Surface-Conditioned Computation: Probing Whether Open LLMs Compute or Retrieve on Physics Problems

Anonymous Authors

Abstract

We ask whether open-weight language models actually *compute* when they answer multi-step physics problems, or whether they pattern-match against memorised solutions. Rather than reporting a single accuracy number, we combine five probes on SciBench physics (323 problems), a multiple-choice control (MMLU physics, 488 problems) and a second free-response benchmark (OlympiadBench physics, 351 problems). Three findings carry the paper. First, *prompting outweighs scale*: chain-of-thought multiplies accuracy by 2.28–2.18× across the Qwen2.5 family, whereas doubling parameters from 7B to 14B adds only 4.6 points, and the benefit is family-dependent rather than universal (1.29× for Llama-3.1-8B). Second, *knowledge is present but execution is not*: the same 7B model answers 66.6% of multiple-choice physics items but only 21.1% when it must produce the number itself. Third, *answers are surface-conditioned*: a paraphrase that leaves physics and data bit-identical changes 30.4% of final answers in every model we test, yet moves only 5 of 240 items across the correct/incorrect boundary, so the instability is value-level wandering rather than a systematic difficulty change. Two further probes constrain the mechanism. Corrupt-and-restore causal tracing localises recoverable behaviour to an early-middle band (roughly layers 4–16 of 28) that we can identify as a range but *not* as a single peak layer, and separating output execution from correctness shows that parseability is layer-invariant while correctness is not; on problems the model cannot solve, restoration never rescues them at any single layer. Error annotation cross-checked by a second, cross-family judge indicates that arithmetic rather than physical knowledge dominates the failures, but agreement between the judges is only $\kappa = 0.133$, so we claim only annotator-robust statements and release the dual-annotated sample as a resource for LLM-as-a-judge research. All experiments ran on a single 80 GB GPU in about 2.6 GPU-hours.

1 Introduction

Chain-of-thought prompting [13] is routinely credited with giving language models "reasoning" ability, and physics benchmarks such as SciBench [12] and OlympiadBench [4] are used to demonstrate progress. A single accuracy number, however, cannot distinguish two very different internal situations: a model that derives the answer from the stated quantities, and a model that recognises the problem family and emits a plausible-looking number.

A binary framing, deliberately, and how the data refine it. We organise the study around the question "compute or retrieve?" because it generates falsifiable predictions: a retrieving model should be insensitive to the numerical values and highly sensitive to surface form, whereas a computing model should show the opposite pattern. We stress in advance that our results do *not* support either pure endpoint. What the probes jointly indicate is a third regime, which we call *surface-conditioned computation*: the stated values do enter the computation (so it is not lookup), the computation is nevertheless highly sensitive to semantically irrelevant phrasing, and it fails systematically at the numerical-execution stage. The title's question is the entry point of the investigation, not its conclusion; §5 returns to this framing explicitly.

We ask three questions and answer them with measurements that are cheap, deterministic and reproducible on a single GPU:

- **What does performance look like?** (§4.1) Five open-weight models on 323 SciBench physics problems, with bootstrap confidence intervals and per-sub-domain breakdowns.

- **Is the model computing or retrieving?** (§4.2) Three perturbations that are semantically different but superficially similar: change the numbers, rewrite the wording, exchange two physical quantities.
- **If it computes, where?** (§4.3) Corrupt-and-restore causal tracing, followed by a separation of "producing a number" from "producing the right number".

A multiple-choice control (§4.4) holds knowledge fixed and varies only the answer format; an error-type analysis with a cross-family annotator check (§4.5) asks *which* stage fails and how reliable such attributions are; and §4.6 tests whether prompt-level scaffolding can repair the instability.

Contributions.

1. A multi-probe protocol that separates computation from retrieval using only black-box generation plus, for one open-weight model, residual-stream interventions — together with the finding that *paraphrase instability* is a reproducible, model-generic signature of surface-conditioned answering, and that it is value-level rather than correctness-level.
2. A representation-vs-output decomposition: output execution is layer-invariant while correctness is layer-localised, and failures on problems the model cannot solve are *not* recoverable by restoring any single layer.
3. An honest measurement result: with a same-family and a cross-family judge annotating the same 495 errors, agreement is only $\kappa = 0.133$. We claim only annotator-robust statements and release the dual-annotated sample as a resource.

2 Related Work

Physics and science reasoning benchmarks. SciBench [12] collects college-level textbook problems with free-response answers; we use its physics subsets and restrict to the 323 problems whose reference answers are numerically gradable. OlympiadBench [4] contains competition problems; only 43% of its physics items have numerically gradable references, a coverage limit we carry through every table. MMLU [5] provides multiple-choice knowledge questions and serves here as a format control rather than as a target.

Causal tracing and localisation. Activation- and state-level interventions are standard for localising behaviour in open models [8, 11]. We use the simplest variant — corrupt the input, then restore the clean residual stream at a single layer — because it requires no training and yields a per-layer curve. In factual-recall studies the recoverable signal typically sits in the middle of the network; our plateau sits *earlier* than the middle band of a 28-layer model, which we discuss in §4.3.

Sensitivity and robustness of LLM answers. Prompt sensitivity is well documented [10], usually as a nuisance to be averaged out. We instead treat sensitivity as the measurement, and — unlike most prior work — we separate *value* changes from *correctness* changes, which turn out to have very different magnitudes.

LLM-as-a-judge and error taxonomies. Using a strong model to label errors is common practice [14], and its reliability is usually reported as agreement with human labels. We contribute a case where two *LLM* judges disagree substantially on a subjective taxonomy, and argue that the disagreement is itself the informative quantity.

3 Method

3.1 Benchmarks and grading

All correctness decisions are made by symbolic/numeric comparison. The final answer is extracted by a precedence-ordered procedure (`ANSWER:` line, taking the last occurrence; then `\boxed{}`; then "answer is X"; then the last number on the last line), normalised for Unicode minus signs, superscripts and thousands separators, and compared to the reference under a 5% relative tolerance (0.01 absolute for near-zero targets).

Reference answers are parsed with a *stricter* parser than model outputs: a reference that is not a single number is marked unscorable and removed from the denominator rather than counted as a model error, because a permissive parser reads expressions such as $x^2 + 2x$ as 2 and silently corrupts the denominator. For SciBench physics all 323 references are numerically gradable, so the denominator is the full set; for OlympiadBench only 152 of 351 are. Grading is covered by 23 self-test cases and 84 unit tests, and an oracle check (feed the reference as the model output) yields 100% self-consistency. Full prompt templates are given in Appendix A.

3.2 Models and inference

We evaluate five open-weight instruct models: Qwen2.5-3B/7B/14B-Instruct [9], Llama-3.1-8B-Instruct [2], and DeepSeek-R1-Distill-Qwen-7B [3]. Decoding is greedy with a fixed seed, so every reported number is deterministic. We use the tokenizer's chat template and left padding for batched generation. All runs use the `transformers` backend: vLLM was *not installed* in our execution environment, so the code path that prefers vLLM never activated and no result mixes backends. Under greedy decoding accuracy is backend-independent; only wall-clock differs (§6).

3.3 Perturbation probes

Each probe is a deterministic transformation of the problem text:

- **numeric**: scale physical values (physics intact, at most two numbers changed, exponents and power bases protected);
- **wording**: paraphrase the framing only, leaving physics and data bit-identical. This is the only variant whose reference answer remains valid, so it is the only one for which we report a paired correctness comparison;
- **swap**: exchange two distinct physical quantities, producing a physically inconsistent problem.

Because the correct answer to a *numeric* or *swap* variant is not directly computable, those variants are scored by the distribution of output change, and we report median and p90 rather than the mean (the sensitivity score $|\Delta|/(|base| + \epsilon)$ is dominated by outliers when the base answer is near zero). The *swap* probe covers only 36 of 323 problems, so its percentages are reported for completeness and carry no quantitative weight.

3.4 Causal tracing

The numeric quantities of a question are replaced by a semantically empty placeholder; the corrupted prompt is run; then the clean residual stream is *restored* at exactly one layer at a time, all other layers remaining corrupted. Restoration is applied during prefill only and never to decode steps, and clean and corrupted sequences are aligned from the right because they tokenise to different lengths. Only problems whose clean run was correct *and* whose corrupted run lost score enter the recovery rate, which we report together with the skip counts.

4 Results

The stages below correspond to the project's Week 1–4 work plan (main table, perturbation probes, causal tracing, error attribution), plus two controls added during revision.

4.1 How large is the prompting effect relative to scale?

Table 1 reports SciBench physics accuracy for all five models under both prompting conditions, with bootstrap 95% confidence intervals.

The effect of prompting exceeds the effect of doubling parameters, though the two are measured on different scales (a ratio versus a difference in points): for the Qwen2.5 family chain-of-thought multiplies accuracy by 2.28×, 2.43× and 2.18× at 3B, 7B and 14B, while going from 7B to 14B adds 4.6 points

Table 1: Accuracy (%) on SciBench physics (323 problems). All reference answers are numerically gradable and all model outputs were parseable, so the denominator is the full 323. Bracketed intervals are bootstrap 95% CIs (10,000 resamples, seed=42) on overall accuracy; sub-domain cells are point estimates. "cot" = zero-shot chain-of-thought, "direct" = answer-only.

Model	Mode	Overall (95% CI)	Sub-domain accuracy (%)					
			Math	Mech	Thermo	Optics	Quantum	General
Qwen2.5-3B	cot	12.1 [8.7, 15.8]	20.0	5.4	12.1	10.6	18.2	9.9
Qwen2.5-3B	direct	5.3 [3.1, 7.7]	2.0	7.1	7.6	6.4	6.1	2.8
Qwen2.5-7B	cot	21.1 [16.7, 25.7]	34.0	21.4	21.2	17.0	15.2	16.9
Qwen2.5-7B	direct	8.7 [5.6, 11.8]	8.0	10.7	6.1	8.5	12.1	8.5
Qwen2.5-14B	cot	25.7 [21.1, 30.7]	40.0	26.8	24.2	23.4	21.2	19.7
Qwen2.5-14B	direct	11.8 [8.4, 15.5]	6.0	17.9	7.6	10.6	30.3	7.0
Llama-3.1-8B	cot	9.9 [6.8, 13.3]	14.0	8.9	12.1	6.4	6.1	9.9
Llama-3.1-8B	direct	7.7 [5.0, 10.8]	6.0	7.1	6.1	8.5	18.2	5.6
R1-Distill-7B	cot	11.1 [7.7, 14.6]	12.0	7.1	10.6	8.5	18.2	12.7

Table 2: Sensitivity of final answers to hard vs. surface perturbations (SciBench physics, 323 base problems, answer-only prompt). *numeric* changes physical values with physics intact; *wording* only rewrites surface phrasing (physics and data identical); *swap* exchanges two distinct physical quantities. "Changed" is the share of answers that differ from the base run. Median / p90 are reported instead of the mean: the sensitivity score is $|\Delta|/(|base| + \epsilon)$ and is dominated by outliers when $|base| \approx 0$.

Model	Variant	Coverage	n parsed	Median sens.	p90 sens.	Changed
Qwen2.5-3B	numeric	305/323 (94.4%)	305	0.500	6.32	77.0%
	wording	240/323 (74.3%)	240	0.000	0.91	33.8%
	swap	36/323 (11.1%)	36	0.840	65.25	83.3%
Qwen2.5-7B	numeric	305/323 (94.4%)	305	0.587	8.96	77.7%
	wording	240/323 (74.3%)	240	0.000	0.95	30.4%
	swap	36/323 (11.1%)	36	0.371	15.06	69.4%
Qwen2.5-14B	numeric	305/323 (94.4%)	305	0.633	7.60	83.9%
	wording	240/323 (74.3%)	240	0.000	0.68	31.7%
	swap	36/323 (11.1%)	36	0.533	6.50	80.6%
Llama-3.1-8B	numeric	305/323 (94.4%)	303	0.082	5.11	57.4%
	wording	240/323 (74.3%)	238	0.000	1.00	24.4%
	swap	36/323 (11.1%)	36	0.335	1.83	63.9%

(21.1% → 25.7%). The prompting benefit is family-dependent rather than a law of nature: for Llama-3.1-8B the same comparison gives only 1.29× (9.9% vs 7.7%), and family can outweigh size — the 3B Qwen model (12.1%) outperforms the 8B Llama model (9.9%). The reasoning-distilled model (DeepSeek-R1-Distill-Qwen-7B, 11.1%) does not beat its same-size instruct sibling, which we attribute to friction between long reasoning traces and this benchmark's terse numeric answer format; we did not run it answer-only, because its instruction-tuned behaviour emits reasoning traces regardless of the prompt and a *direct* comparison would not be like-for-like.

4.2 Is the model computing or retrieving?

Table 2 reports the three perturbation probes for four models.

Changing the numbers moves the answer most of the time (77.7% for Qwen2.5-7B), so the stated values do participate in the computation: the model is not a lookup table. A paraphrase that leaves physics and data bit-identical changes 30.4% of answers, and the pattern repeats across models (33.8%, 31.7%, 24.4%); Figure 1 visualises the contrast.

Crucially, "changing the answer" is not the same as "changing correctness". Table 3 scores the paraphrase variants against the reference and reports the paired 2×2 counts: for Qwen2.5-7B only 5 of 240 items cross the correct/incorrect boundary (1 correct → wrong, 4 wrong → correct; exact McNemar $p = 0.375$), and the pattern is the same for the other three models. We therefore state the weaker and better-supported claim:

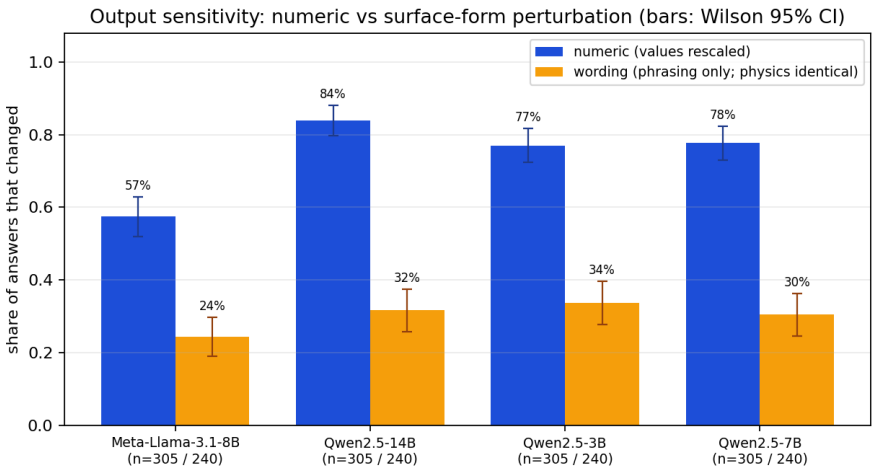


Figure 1: Share of final answers that change when physical values are rescaled (*numeric*) versus when only the phrasing is rewritten (*wording*). 305 and 240 items respectively, out of 323. The paraphrase bar is non-zero for every model, which is the paper's central robustness observation.

Table 3: Paraphrase variants scored for *correctness*: 2×2 paired counts. The perturbation rewrites phrasing only, so the reference answer stays valid and a paired comparison is meaningful. Very few items cross the correct/incorrect boundary, and the two directions are of similar size, i.e. we do not detect a systematic shift (exact McNemar test on the discordant pairs).

Model	both correct	correct→wrong	wrong→correct	n discordant	exact p
Qwen2.5-3B	8	5	5	10	1.000
Qwen2.5-7B	19	1	4	5	0.375
Qwen2.5-14B	27	1	6	7	0.125
Llama-3.1-8B	19	1	0	1	1.000

240 items per model (the paraphrase coverage). A value can change without changing correctness: the much larger 'changed answer' rate in Table 2 counts value changes, whereas this table counts only changes that cross the correctness boundary.

we detect no systematic shift in correctness, and the discordant pairs are roughly balanced. The paraphrase instability is real, but it is *value-level wandering* rather than a systematic loss of competence — a distinction that a single accuracy number hides entirely.

4.3 Where is the computation, and is the output on the same pathway?

Table 4 locates the computation in depth.

Scope. Of 150 attempted problems, only 14 satisfy the strict inclusion rule (134 skipped as clean-wrong, 2 as corruption-flat). This subset is strongly selected: it consists of problems the model *already solves*. Every statement in this subsection is therefore scoped to solvable problems, not to the benchmark as a whole.

A band, not a peak. Recovery rises across the early-middle layers and collapses after layer 18 (Figure 2). The point estimate is maximal at layer 8 (+0.84), but the bootstrap 95% CI on that mean is $[+0.69, +0.95]$ and a paired bootstrap over problems cannot separate it from 6 other layers inside the plateau. We therefore claim a recoverable *range* (roughly layers 4–16) rather than a single peak layer; only layers beyond ∼18, and the embedding region (whose variance is an order of magnitude larger), are clearly distinguishable.

Relation to factual-recall localisation. In factual-recall tracing the recoverable signal typically occupies the middle of the network. Our plateau (layers 4–16 of 28, i.e. roughly the first half) sits *earlier* than that.

Table 4: Causal tracing on Qwen2.5-7B-Instruct (28 layers): recovery rate when the clean residual stream is restored at a single layer of the corrupted run. Only problems whose clean run was correct *and* whose corrupted run actually lost score are counted ($n = 14$ of 150 attempted; skipped 134 clean-wrong and 2 corrupted-flat), so the estimate applies to problems the model can already solve. Intervals are bootstrap 95% CIs over problems (10,000 resamples, seed 42).

Layer range	Mean recovery	Bootstrap 95% CI	±1 std across problems
layers 0–2	+0.431	[-0.11, +0.84]	±0.912
layers 4–8[†]	+0.665	[+0.33, +0.95]	±0.378
layers 10–16	+0.614	[+0.28, +0.88]	±0.431
layers 18–22	+0.238	[+0.00, +0.64]	±0.337
layers 24–26	+0.061	[+0.00, +0.19]	±0.174

[†]Recovery is maximal at layer 8 (+0.837, 95% CI [+0.69, +0.95]), but a paired bootstrap over problems *cannot* separate it from 6 other layers inside the plateau. We therefore claim a recoverable *range* (roughly layers 4–16), not a single peak layer; only layers beyond ∼18 and the embedding region (whose variance is an order of magnitude larger) are clearly distinguishable.

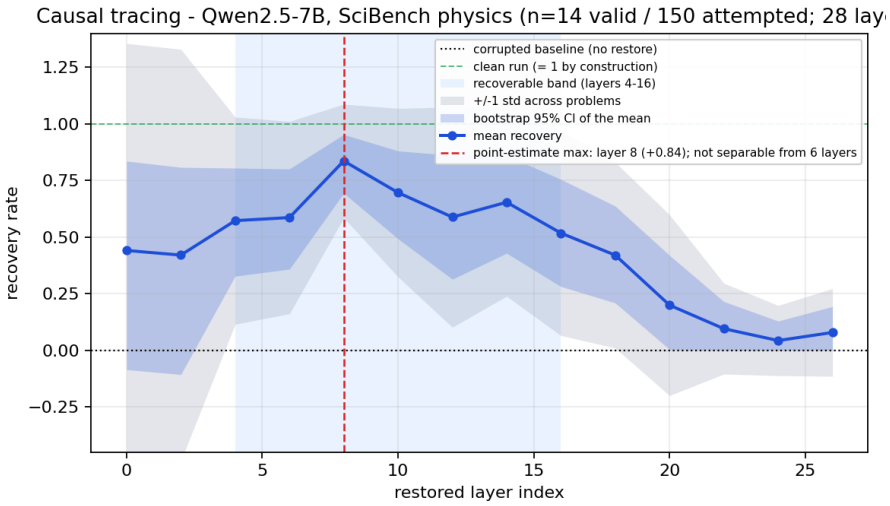


Figure 2: Recovery rate as a function of the restored layer, with ±1 standard deviation across problems. The dotted line at zero is the corrupted (no-restore) baseline, so the height above it is what restoration buys. Behaviour is recoverable over an early-middle band.

Two differences plausibly explain the discrepancy. First, our corruption replaces *numerical quantities* with a placeholder, whereas factual-recall protocols replace a *named subject*; collapsing the numeric input removes information that has to be re-encoded early, so restoration must intervene early. Second, the target here is a derived scalar rather than a retrieved attribute, so the computation is spread over more layers. These are hypotheses consistent with our data, not findings we can establish without additional corruption types.

Separating execution from semantics. Restoring a clean state is only defined for problems whose clean run is correct, so the recovery curve above cannot tell us whether restoration could *rescue* failures. We therefore instrument all 100 attempted problems and record two outcomes at every restore point: whether the model emits a parseable number (output execution) and whether that number is correct (semantics). Table 5 shows the result, split by clean-run correctness.

Parseability is essentially layer-invariant: 96% even for the corrupted baseline, and 97–100% at every restore point. The model always emits a number; corruption never makes it decline. Correctness, by contrast, depends on the restored depth: for the 10 problems the clean run solves, restoration recovers 7 of them (70%, Wilson 95% CI [40, 89]) at the best layer — suggestive but imprecise at this sample size; for the 90 problems it never exceeds 3.3% at any layer. The two pathways are therefore distinct, and the failures

Table 5: Output execution versus semantic correctness. The clean residual stream is restored at one layer of the corrupted run; we record whether the model emits a parseable number (*parse*) and whether that number is correct (*acc*). Parseability is essentially layer-invariant, whereas correctness moves with the restored depth — and only for problems the model can solve without corruption. Small brackets are Wilson 95% binomial intervals; with $n = 10$ in the left group they are wide, so the recovery of that group is indicative rather than precise.

Condition	Parse rate	Accuracy [Wilson 95% CI]	
		clean correct (n=10)	clean wrong (n=90)
corrupted (no restore)	96.0%	–	1.0%
clean (no corruption)	100.0%	–	10.0%
restore layer 0	99.0%	70.0 [40, 89]%	1.1 [0, 6]%
restore layer 4	99.0%	40.0 [17, 69]%	3.3 [1, 9]%
restore layer 8	99.0%	50.0 [24, 76]%	0.0 [0, 4]%
restore layer 12	97.0%	70.0 [40, 89]%	0.0 [0, 4]%
restore layer 16	100.0%	60.0 [31, 83]%	3.3 [1, 9]%
restore layer 20	100.0%	30.0 [11, 60]%	2.2 [1, 8]%
restore layer 24	97.0%	10.0 [2, 40]%	0.0 [0, 4]%

Qwen2.5-7B-Instruct, 100 attempted problems, 7 restore points (every 4 layers of 28). Parse rate stays within 97.0–100.0% at every layer, including the corrupted baseline: the model always emits a number. Correctness, by contrast, is only recoverable for problems the clean run solves, and never recovers for the rest at any single layer.

Table 6: Same model, different answer formats. Multiple-choice recognition is easy; producing the final number is not. Task formats are not directly comparable (selection vs. generation), so the contrast is interpreted qualitatively.

Task	Answer format	n (gradable)	Accuracy
MMLU physics (high-school + college + conceptual)	select A–D	488	66.6 [62.3, 70.7]
SciBench physics, cot	write the number	323	21.1 [16.7, 25.7]
SciBench physics, direct	write the number	323	8.7 [5.6, 11.8]
OlympiadBench physics, cot	write the number	152 of 351	23.7 [17.1, 30.3]

Model: Qwen2.5-7B-Instruct. OlympiadBench is evaluated only on the subset whose reference answer is numerically gradable (152 of 351), which is biased towards easier items; it must not be compared numerically against SciBench.

are capability limits rather than misplaced representations that a patch could repair.

4.4 Knowledge or execution? A format control

Table 6 holds the model and the subject matter fixed and varies only the answer format.

The same Qwen2.5-7B gets 66.6% of 488 multiple-choice physics items right but only 21.1% when it must produce the number itself. Since the multiple-choice items test the same physics, missing knowledge cannot be the explanation: the knowledge is retrievable, the numerical execution is not.

OlympiadBench is used *only* to check the direction of the scale trend: accuracy rises from 23.7% to 30.3% (+6.6 points), consistent with SciBench. Because its 43% gradable coverage is biased towards items with clean scalar answers, we make no absolute-level claim from this benchmark and never compare its values numerically against SciBench; per-sub-domain counts are so small (six gradable optics items, all incorrect) that we do not interpret them.

4.5 Which stage fails, and can we tell reliably?

We annotated every SciBench chain-of-thought error with a three-way scheme — E1 wrong physical law, E2 algebraic/arithmetic/unit error, E3 violated constraint — and repeated the annotation with a second, cross-family judge on the same samples.

Under *both* judges E2 is the modal category and E3 stays small, and the 7B/14B distributions do not differ significantly under either labelling (same-family $\chi^2 = 4.55$, $p = 0.208$; cross-family $\chi^2 = 1.09$, $p = 0.778$).

Table 7: Error-type distribution of the SciBench cot runs, annotated independently by a *same-family* judge (Qwen2.5-14B) and a *cross-family* judge (Llama-3.1-8B); the two blocks are separated by the midrule. *unscorable_ref* (reference answer not numerically gradable) and *unparsed_pred* (no parseable answer) samples are excluded. E1 = wrong physical law; E2 = algebraic / arithmetic / unit error; E3 = violated constraint / internal inconsistency; N = grader false negative.

Model	Judge	Errors	E1	E2	E3	N
Qwen2.5-7B	Qwen2.5-14B (same family)	255	29.4%	57.6%	10.2%	2.7%
Qwen2.5-14B	Qwen2.5-14B (same family)	240	22.5%	65.0%	8.3%	4.2%
Qwen2.5-7B	Llama-3.1-8B (cross family)	255	9.0%	82.0%	9.0%	0.0%
Qwen2.5-14B	Llama-3.1-8B (cross family)	240	7.5%	85.4%	7.1%	0.0%

The 7B/14B comparison is run under *both* judges: same-family $\chi^2 = 4.55$, $df = 3$, $p = 0.208$; cross-family $\chi^2 = 1.09$, $p = 0.778$ — no significant difference either way. Agreement between the two judges is only $\kappa = 0.13$ (60% raw agreement, $\kappa_w = 0.12$ linearly weighted). We therefore claim only what survives both annotators: the modal category is E2 under both judges and E3 stays small; the precise E1/E2 split is *not* claimed, since the judges differ by a factor of three on E1.

Table 8: Prompt scaffolding as a substitute for capability. The same 150 problems are evaluated with a matched budget (2048 new tokens) under two prompts: standard chain-of-thought, and a structured prompt that additionally asks the model to write down the knowns and unknowns explicitly. Accuracy does not improve significantly, so the instability documented in Table 2 is not a missing-scaffold problem.

Prompt	Accuracy	95% CI	No `ANSWER:`	Output chars (mean / max)
standard cot	24.0%	[17.3, 31.3]	71%	2377 / 4224
structured (knowns/unknowns)	26.0%	[19.3, 33.3]	90%	2373 / 4639
paired difference: +2.0 pp, McNemar $p = 0.700$				

The last column exposes a separate limitation: most outputs ignore the requested `ANSWER:` format and are graded through the fallback extractor. Output lengths are well below the token budget, so this is a format-compliance failure rather than truncation.

The larger model therefore gets more problems right rather than failing differently, which matches the format control: the bottleneck is execution, not physical knowledge.

Agreement between the judges is, however, only $\kappa = 0.133$ (60% raw agreement, $\kappa_w = 0.118$ linearly weighted). The per-class picture explains why: on E2 the judges agree 88% of the time, on E1 only 16%. Applying the wrong law usually also produces wrong arithmetic, so the observable text is consistent with either label. We consequently report only the annotator-robust statement (modal category E2, small E3, no 7B/14B difference) and treat the disagreement as a finding about error taxonomies rather than as an inconvenience (§5).

4.6 Can prompt scaffolding repair the instability?

If the paraphrase instability were caused by a missing solution scaffold, telling the model to write down the knowns and unknowns explicitly should help. At a matched token budget on the same 150 problems it changes accuracy from 24.0% to 26.0% (+2.0 points, McNemar $p = 0.700$) — no reliable gain (Table 8).

4.7 An output-format ceiling

One limitation deserves separate mention because it caps what any strict-format evaluation can see. Between 71% and 90% of outputs never emit the requested `ANSWER:` line and are graded by the fallback extractor ("last number on the last line"). Output lengths stay far below the token budget, so this is a format-compliance failure rather than truncation, and accuracy is statistically indistinguishable between compliant and non-compliant outputs (Table 8) — but an evaluation that insisted on the format would report a lower ceiling than ours.

5 Discussion

From a binary question to a third regime. We opened with "compute or retrieve?" because it makes falsifiable predictions. The data reject both endpoints and support a third regime. A pure retrieval story predicts insensitivity to numerical values and high sensitivity to surface form; a pure computation story predicts the opposite. We observe *both* sensitivities at once: rescaled values move 77.7% of answers (so the values are used), and a semantics-preserving paraphrase moves 30.4% (so surface form is also used). The additional separation we introduce — counting only changes that cross the correct/incorrect boundary — shows that the paraphrase effect is *value-level*: just 5 of 240 items change correctness, with discordant pairs roughly balanced. We therefore name the regime *surface-conditioned computation*: the model computes, its computation depends on irrelevant phrasing, and its outputs wander within a band that mostly keeps correctness fixed. Calling this "reasoning failure" or "retrieval" would both misdescribe it.

What the evidence jointly supports. Five probes converge on one reading. (i) Accuracy on SciBench physics tops out at 25.7% for the largest model we ran and gains only 4.6 points from a two-fold parameter increase, while chain-of-thought multiplies accuracy by 2.28–2.18× for the Qwen family — prompting outweighs scale in this setting. (ii) The same model reaches 66.6% on multiple-choice MMLU physics but only 21.1% when it must produce the number itself, which rules out missing knowledge: it is present and retrievable, while numerical execution is not. (iii) The paraphrase instability described above is model-generic (33.8%, 31.7%, 24.4% in the other three models) and is value-level rather than correctness-level. (iv) Restoring clean residual states recovers behaviour over an early-middle band (layers 4–16 of 28) that we can localise as a range but not as a single peak, and the recovery collapses after layer 18. (v) Separating output execution from correctness shows parseability is layer-invariant (96% even under corruption) while correctness is depth-dependent, and on problems the model cannot solve restoration never exceeds 3.3% at any single layer. The failures are capability limits, not misplaced representations that a layer patch could repair.

Why the error taxonomy is itself uncertain (and why that is a contribution). Our cross-annotator agreement ($\kappa = 0.133$, $\kappa_w = 0.118$ linearly weighted, 60% raw agreement over 495 samples) is low, and this is not merely a nuisance for our own analysis — it reveals a fundamental limitation of the taxonomy itself. The per-class breakdown is diagnostic: the judges agree 88% of the time on E2 but only 16% on E1. That is exactly what one expects if the boundary between "misapplying a physical law" and "algebraic miscalculation" is intrinsically ambiguous: when a model applies the wrong law, the resulting arithmetic is wrong too, so the observable text is consistent with either label. We therefore report only the coarse, annotator-robust statements — E2 is the modal category under both judges, E3 is rare, and 7B/14B do not differ under either labelling ($p = 0.208$ same-family, $p = 0.778$ cross-family) — and deliberately do not quote precise category shares. This suggests that evaluating reasoning purely through coarse error categories is insufficient: taxonomy-based analysis inherits an annotator-dependent ceiling that stays invisible unless agreement is measured. We release our dual-annotated sample (495 wrong answers from two models, each labelled independently by a same-family and a cross-family judge) as a resource not only for physics reasoning but for research on LLM-as-a-judge attribution, where the *disagreement* between judges is the signal rather than noise.

Implications for mitigation. Two findings are directly actionable. First, since the dominant failure mode is arithmetic rather than physical (E2), the highest-leverage interventions are execution-side — tool use, code interpreters, or explicit symbolic bookkeeping — rather than more physical pre-training data. Second, since 30.4% of answers move under a semantics-preserving paraphrase, single-prompt evaluation understates the variance of a model's competence; reporting paraphrase-consistency alongside accuracy is a cheap and informative robustness metric. We also tested whether the instability is recoverable at inference time by adding structural scaffolding to the prompt: at a matched token budget the change is +2.0 points (McNemar $p = 0.700$), so prompt-level scaffolding does not substitute for capability, which further argues that the mitigation has to be execution-side rather than wording-side.

Threats to validity. All numbers come from a single GPU class (A800-80G) and a `transformers` backend without continuous batching, so wall-clock times are not comparable to a vLLM deployment, although greedy decoding makes accuracies backend-independent. Causal tracing covers one open-weight model and the strict-inclusion subset (14 of 150 attempted), so the layer localisation is a qualitative early-middle pattern rather than a precise per-layer estimate; the layer-parse analysis rests on 10 correct-baseline problems, which we accompany with Wilson intervals. OlympiadBench results refer to the 43% of problems with numerically gradable references and are used only for the direction of the scale trend. Error-type shares are annotator dependent ($\kappa = 0.133$) and are reported coarsely. All models are evaluated zero-shot with a single fixed prompt per condition; prompt sensitivity is itself one of the phenomena measured, so no per-model prompt tuning is attempted.

6 Limitations

Statistics of the causal probe. The causal analysis rests on 14 strictly-included problems, and the layer-parse analysis on 10 correct-baseline problems. Both are small: we report bootstrap intervals for the recovery curve and Wilson intervals for the small proportions, we do not claim a single peak layer, and we scope the conclusions to problems the model can already solve.

Annotator dependence. Error-type shares are annotator-dependent ($\kappa = 0.133$, $\kappa_w = 0.118$) and are reported coarsely for that reason. The cross-family judge is still an LLM, so this is a check against family-specific bias, not a substitute for human annotation.

Benchmark coverage. OlympiadBench numbers refer to the 43% of problems with numerically gradable references and are used only for the direction of the scale trend.

Backend and hardware. All runs use a single A800-80G with the `transformers` backend and no continuous batching, so wall-clock times are not comparable to a vLLM deployment; greedy decoding makes accuracies backend-independent.

Probe construction. The paraphrase probe uses deterministic rule-based rewriting; a model-generated paraphrase would be stronger but introduces rewrite quality as a confound. The scaffolding probe used one prompt design at one budget, so its null result is evidence against that intervention, not proof of impossibility. Finally, all models are evaluated zero-shot with a single fixed prompt per condition; prompt sensitivity is itself one of the phenomena measured, so no per-model prompt tuning is performed.

A Prompt templates

All prompts are rendered from a single template table in `src/prompts.py`; the MCQ templates are selected automatically when the question contains at least two options of the form "A. . . . ". Every template demands the final answer on its own line prefixed with `ANSWER:`, which is what the extractor looks for first.

Cot / free-response

```
Solve the following physics problem step by step. Show all work.
At the end, give the final numeric answer on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Cot / multiple-choice

```
Solve the following multiple-choice physics problem step by step. Show all work.
At the end, give the chosen option letter on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Direct / free-response

```
Answer the following physics problem directly with a single number (no explanation).
Give the final answer on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Direct / multiple-choice

```
Answer the following multiple-choice physics problem.
Give the chosen option letter on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Structured / free-response

```
Solve the following physics problem step by step.
Before solving, first write down the knowns and unknowns explicitly, then derive the solution.
Show all work. At the end, give the final numeric answer on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Structured / multiple-choice

```
Solve the following multiple-choice physics problem step by step.
Before solving, first write down the knowns and unknowns explicitly, then compare the options.
At the end, give the chosen option letter on its own line, prefixed with "ANSWER:".
Problem: <problem statement>
```

Chat template The rendered prompt is then wrapped by the tokenizer's chat template (`apply_chat_template`), so the model sees its own instruction format. Whether a chat template was applied is recorded in every result file as `meta.chat_template`. Example problem used in this paper: `A 2.0 kg block slides down a 30 degree incline at 3.0 m/s. ...`

B Result-file schema

Every evaluation writes one JSON file. The top level records the run identity and provenance; `detail` holds one record per problem, which is what the audit tables are built from. Minimal schema of `results/eval_*.json`:

```
"model": string,
"benchmark": string,
"mode": string,
"n": int,
"n_scorable": int,
"n_unscored": int,
"n_unparsed_pred": int,
"overall_acc": string,
"per_domain_acc": object,
"detail": [ ... 323 records ... ],
"wall_time_s": float,
"max_new_tokens": int,
"data_composition": object,
"prompt_fingerprint": string,
"meta": object,
# detail[i]:
#   "id": string,
#   "question": string,
#   "domain": string,
#   "subset": string,
```

```
#   "ref": string,
#   "pred_raw": string,
#   "pred_val": float,
#   "correct": bool,
#   "status": string,
#   "scorable": bool,
#   "n_pred_chars": int,
```

Two fields deserve emphasis. `scorable` marks whether the *reference* answer could be parsed; rows with `scorable=false` are excluded from every denominator and reported as `n_unscored`. `status` is one of `correct`, `incorrect`, `unparsed_pred` or `unscorable_ref`, so a reader can recompute any accuracy in this paper from the raw file without re-running a model. Sensitivity files (`sensitivity_*_*.json`) additionally carry `items[]`, one entry per perturbed problem with the perturbed question text and both answers, enabling per-item audit of the perturbation.

C Statistical details

Bootstrap. Confidence intervals resample problems with replacement $B = 10{,}000$ times using `random.Random` seeded with 42, and take the 2.5/97.5 percentiles of the resampled statistic (order statistics of the resampled accuracy, i.e. a percentile interval), the standard non-parametric approach of Efron and Tibshirani [1]. The same seed is used for every model, so intervals are exactly reproducible across machines and do not depend on the NumPy version.

Layer-wise interval and paired test. The recovery curve has $n = 14$ problems. Each layer's mean gets a percentile bootstrap interval from the same B; to test whether the point estimate's best layer is meaningful, we bootstrap the *difference* between that layer and every other layer (2,000 resamples) and ask whether the interval excludes zero. With this test the best layer (layer 8) is *not* separable from 6 other layers, which is why the paper claims a band rather than a peak.

Small-sample proportions. Proportions computed on few observations are reported with Wilson score intervals (95%), which behave better than the normal approximation near 0 and 1. For example the layer-parse recovery on the correct-baseline group is a proportion of a 10-problem sample.

Paired tests. Correctness comparisons between two conditions on the same problems use McNemar's test [7] on the discordant pairs. When the number of discordant pairs is small we report the exact binomial two-sided p-value rather than the asymptotic χ^2 version.

Annotator agreement. We report both unweighted Cohen's κ and a linearly weighted κ_w [6] that gives partial credit for adjacent categories, plus per-class agreement. The distinction matters because the categories are heavily imbalanced (E2 dominates), which depresses κ even when most individual decisions agree.

Table 9: Per-class agreement between the same-family and cross-family judge ($n = 495$ samples; counts are per class).

Class	cross-family count	same-family count	both	agreement given same-family label
E1	129	41	20	16%
E2	303	414	267	88%
E3	46	40	11	24%
N	17	0	0	0%

D Reproducibility

Code and version. All code, evaluation scripts, and analysis utilities are released in the accompanying repository (`https://github.com/380992846-tech/EMPR`) at the exact commit used for this paper

(ec5d0c02c35660a3ac83fc27a1b67de9f0b9df6d). Defaults live in a single configuration file (config.yaml); each analysis stage has a single entry point. Every result file records its own provenance in a meta block (git_commit, execution environment, prompt_fingerprint, max_new_tokens, chat_template, wall_time_s), so any number in the paper can be traced to the run that produced it.

Datasets. **SciBench** physics (https://github.com/mandyyyyii/scibench; subjects matter, class, fund, diff, quan, thermo) yields $N_{sci} = 323$ problems, of which mechanics 56, thermodynamics 66, optics 47, math 50, quantum 33, general 71. All 323 reference answers are numerically gradable, so no sample is excluded from the denominator of the main table. **OlympiadBench** physics (https://huggingface.co/datasets/Hothan/OlympiadBench; configs OE_TO_physics_en_COMP and OE_TO_physics_zh_CEE) yields 351 problems, of which only 152 (43.3%) have a numerically gradable reference answer; the remaining 199 are symbolic or multivariate solutions and are excluded from the denominator (reported as n_unscored). **MMLU** physics (cais/mmlu; high-school, college and conceptual physics) yields 488 multiple-choice problems. All datasets are public and are fetched by scripts/download_data.py; SciBench is loaded from the local data/ copy when present so that the exact same 323 problems are used on every machine (EMPR_PREFER_HF=1 overrides this).

Models. All models are open-weight and were run from public checkpoints: Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, Meta-Llama-3.1-8B-Instruct, and DeepSeek-R1-Distill-Qwen-7B. No closed-weight or API-only model contributes to any reported number. Causal tracing is performed exclusively on an open-weight model whose residual stream is accessible (Qwen2.5-7B-Instruct). Models listed in config.yaml but not reported here were not run and no claim is made about them.

Inference settings. Decoding is **greedy** (temperature = 0) with a fixed seed (42), so all outputs are deterministic. Maximum new tokens were 1024 for chain-of-thought prompts (1536 for the reasoning-distilled model, whose chains are longer) and 96 for answer-only prompts (64 for multiple-choice and for the sensitivity probes). Inference used the transformers backend on a single GPU in bfloat16 with batches of 16–48; vLLM was not available in the execution environment, so the transformers path was used throughout and no result mixes backends. The tokenizer's chat template is applied to every instruct model (recorded as meta.chat_template) and batch generation uses left padding.

Answer extraction and grading. Correctness is judged by symbolic/numeric comparison. The final answer is extracted by a precedence-ordered procedure (ANSWER: line, taking the *last* occurrence; then \boxed{}; then "answer is X"; then the last number on the last line), normalised for Unicode minus signs, superscripts, \times and thousands separators, and compared to the reference under a relative tolerance of 5% (absolute tolerance 0.01 for near-zero targets) via the Python/SymPy evaluator. Reference answers are parsed with a *stricter* parser than model outputs: a reference that is not a single number (up to units) is marked unscorable_ref and removed from the denominator rather than counted as a model error, because a permissive parser would read x^2+2x as 2 and silently corrupt the denominator. Grading is covered by 23 self-test cases (python -m src.grading, 23/23 pass) and by the unit-test suite (84 tests). As an end-to-end check, using the reference answers as model outputs yields 323/323 self-consistent judgements. No LLM judge is used for the primary results.

Statistical protocol. All reported accuracies carry bootstrap 95% confidence intervals from 10,000 resamples with seed = 42. The bootstrap is implemented in pure Python using random.Random and order statistics, so the intervals are reproduced *identically* across machines, independent of the NumPy version. Pairwise comparisons use McNemar's test (on paired per-problem outcomes); error-distribution comparisons use an uncorrected Pearson chi-square; agreement between the two error annotators is quantified with Cohen's κ.

Sensitivity probes. Three deterministic perturbations are applied to the same 323 SciBench problems: *numeric* (scale physical values; physics intact; 305/323 problems affected), *wording* (surface paraphrase only, physics and data bit-identical; 240/323), and *swap* (exchange two distinct physical quantities, creating a physically inconsistent problem; 36/323). Because the correct answer to a *numeric* or *swap* variant is not

Table 10: Compute breakdown for all reported runs (single A800-80G, transformers backend, bf16, greedy decoding). Times are GPU-busy time; model download and orchestration are excluded.

Stage	Runs / scale	GPU time
SciBench, chain-of-thought	5 models × 323	1.05 h
SciBench, answer-only	4 models × 323	0.02 h
OlympiadBench, chain-of-thought	2 models × 351 (152 gradable)	0.58 h
MMLU physics, multiple choice	1 model × 488	0.002 h
Scaffolding comparison	1 model × 150 × 2 prompts (2048 tok)	0.57 h
Sensitivity probes	4 models × 323 × 3 variants	0.10 h
Causal tracing, valid-only	1 model, 150 attempted (14 valid)	0.05 h
Causal tracing, all-attempts	1 model, 100 attempted × 7 layers	0.08 h
Error annotation	2 judges × 2 models × 495 samples	0.17 h
Total	25 runs	**≈2.6 h**

directly computable, only the distribution of output change is reported for those variants; *wording* keeps the reference answer valid, so it additionally supports a paired accuracy comparison. Median and p90 sensitivity are reported instead of the mean, since the score $|\Delta|/(|base| + \epsilon)$ is dominated by outliers when $|base| \approx 0$.

Causal tracing. We follow the corrupt-and-restore protocol: the numeric quantities of a question are replaced by a semantically empty placeholder (C2 corruption), the corrupted prompt is run, and the clean residual stream is restored at exactly one layer at a time; all other layers remain corrupted, isolating each layer's contribution. Restoration is applied only during prefill (never to single-token decode steps), and the clean and corrupted sequences are aligned from the right because they tokenise to different lengths. Only problems whose clean run was *correct* and whose corrupted run *lost score* enter the recovery rate; of 150 attempted problems, 14 qualify (134 skipped as clean-wrong, 2 as corruption-flat), which is the sample size behind the reported curve.

Hardware and compute. All runs were executed on a single NVIDIA A800-80G PCIe GPU (112 vCPU, 991 GB RAM, 50 GB scratch disk) with bf16 weights. Checkpoints were downloaded one at a time and deleted after use, so peak scratch usage stayed below 30 GB; a single 14B checkpoint needs ≈30 GB free. Total compute for all reported runs is **≈2.6 GPU-hours**, of which ≈2.2 h is the fourteen evaluation runs (summed from the `wall_time_s` recorded in each result file) and the remainder is sensitivity probes, causal tracing, the scaffolding comparison and error annotation. Table 10 gives the breakdown. An additional ≈1.0 GPU-hour was spent on one full-scale causal-tracing run that was aborted (it was too slow to be practical at full layer resolution) and then re-run at reduced scale; it contributes no reported number.

Reproducibility checklist.

- Deterministic decoding (temperature 0, seed 42): ✓
- Public datasets + scripted download, pinned local copy: ✓
- Open-weight models, public checkpoints: ✓
- Symbolic/numeric grading with 23-case self-test and 83 unit tests: ✓
- Reference-answer strictness separated from output-extraction leniency: ✓
- Bootstrap 95% CI, machine-independent (pure Python): ✓
- Causal tracing on an open-weight model only: ✓
- Code, config and per-run metadata at a pinned commit: ✓
- Error-type annotation cross-checked with a second, cross-family judge: ✓

Limitations affecting reproducibility and interpretation. (i) The `swap` perturbation covers only 36/323 problems (11.1%) because many problems contain quantities of a single type or carry no unit; this is reported rather than artificially inflated, and the swap results are treated as supporting evidence only. (ii) OlympiadBench is evaluated only on the 43.3% of problems whose reference answer is numerically gradable; that subset is biased towards easier items, so OlympiadBench accuracies must not be compared numerically with SciBench accuracies. (iii) Error-type shares are annotator-dependent: a same-family judge (Qwen2.5-14B) and a cross-family judge (Llama-3.1-8B) agree on only $\kappa = 0.133$ (60.2% raw agreement) on the same 495 samples. Both judges identify E2 (algebraic/computational error) as the dominant category and E3 as rare, which is what we claim; the precise E1/E2 split is not claimed, because the E1/E2 boundary is intrinsically fuzzy (applying the wrong law usually also shows up as an arithmetic error). (iv) Unparsed model outputs are counted and reported (`n_unparsed_pred`), never imputed; on SciBench this count is zero for all reported runs. A related and more consequential limitation is *format compliance*: between 71% and 90% of chain-of-thought outputs ignore the explicitly requested `ANSWER:` line and are graded through the fallback extractor ("last number on the last line"). Output lengths stay well below the token budget (mean ≈2,400 characters, max ≈4,600 against a 2048-token cap), so this is not truncation, and accuracy does not differ systematically between compliant and non-compliant outputs (0.21 vs. 0.25 in the matched subset), i.e. the fallback does not obviously bias the headline numbers. It nevertheless means that any evaluation relying on strict output formatting would measure a lower ceiling than ours. (v) Throughput figures refer to the transformers backend; vLLM would be 5–10× faster but is not used here, so wall-clock times are not comparable across backends. (vi) Prompt-level scaffolding does not substitute for capability: adding an explicit "write down the knowns and unknowns" instruction changes matched-budget accuracy by +2.0 percentage points (McNemar $p = 0.70$, same 150 problems, 2048-token budget), so we do not claim that prompt engineering repairs the instability documented by the paraphrase probes.

Acknowledgements

[TODO: funding / compute providers / reviewers]

References

[1] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[2] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[3] Daya Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[4] Chaoqun He, Renjie Luo, Yuzhe Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jie Hu, Xu Han, Yujie Huang, Yudi Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.14008.

[5] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2009.03300.

[6] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

[7] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[8] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2202.05262.

[9] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[10] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design, or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.

[11] Alessandro Stolfo, Yonatan Belinkov, and Mrinmaya Sachan. A mechanistic interpretation of arithmetic reasoning in language models using causal mediation analysis. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.15054.

[12] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun Loomba, Shichao Zhang, Yizhou Sun, and Wei Wang. SciBench: Evaluating college-level scientific problem-solving abilities of large language models. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2307.10635.

[13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.